

PROJECT REPORT

SkillOrbit: Concurrency-Safe Event Booking & Management System

Full-Stack Web Development Capstone Project Documentation

Project Title:	SkillOrbit Event Booking & Management System
Curriculum Domain:	Enterprise Full-Stack Web Development Capstone
Developer & Engineer:	Cherukuri Venkata Karthikeya
Architectural Pattern:	Decoupled Client-Server REST Architecture (MERN)
Frontend Technology:	React 19, Vite, Tailwind CSS, Lucide React, Recharts
Backend Technology:	Node.js, Express.js REST API, Multer, json2csv
Database Engine:	MongoDB Atlas Cloud Database (Mongoose 8.9 ODM)
Automated QA Pass Rate:	100% Passed (77 / 77 Integration & Unit Tests)
Live Frontend URL:	https://skillorbit-event-booking-system.vercel.app/
Live Backend URL:	https://skillorbit-backend.onrender.com
Repository:	VenkataKarthikeya-eng/skillorbit-event-booking-system
Submission Date:	October 2026

Executive Verification Summary:

This capstone project report presents the complete design, data modeling, concurrency engineering, security boundaries, administrative telemetry, and verification results of the SkillOrbit platform. All specifications are verified directly against the running production codebase and automated test suites.

2. Abstract

Modern institutions host hundreds of technical conferences, workshops, and symposiums annually. Managing attendee registrations using spreadsheets or basic forms frequently leads to concurrency race conditions, resulting in room capacity overbooking, duplicate tickets, and administrative blind spots. The **SkillOrbit Event Booking System** provides a full-stack MERN architecture engineered to resolve these issues. Key capabilities include public catalog discovery, atomic seat decrements via MongoDB \$gte and \$inc operators, anti-double-booking limits (maximum 10 cumulative tickets per user per event), instant digital boarding pass generation, attendee cancellation with automated inventory rollback, executive Recharts analytics, and one-click RFC 4180 CSV exports. The system has been validated with 77 automated test scenarios, achieving a 100% pass rate.

3. Introduction

Collegiate institutions and technical communities require a dependable, intuitive event registration infrastructure. SkillOrbit addresses this need through a decoupled client-server web architecture built with React 19, Express.js, and MongoDB Atlas. The system provides a seamless booking experience for attendees while equipping organizers with tools to manage capacities, prevent overselling, and analyze registration data in real time.

4. Problem Statement

Traditional registration workflows suffer from four structural vulnerabilities:

- **Concurrency Race Conditions:** High-demand registration periods trigger simultaneous database updates that can exceed physical room capacity.
- **Seat Hoarding & Duplicate Bookings:** Platforms lacking per-attendee reservation limits allow single users to monopolize limited tickets.
- **Absence of Self-Service Cancellation:** Attendees unable to attend have no self-service mechanism to cancel, leaving seats artificially occupied.
- **Lack of Centralized Telemetry:** Organizers lack real-time visibility into occupancy rates and registration velocity across categories.

5. Objectives

- **Zero Overselling:** Enforce atomic MongoDB updates to guarantee room capacity is strictly conserved under concurrent load.
- **Per-User Ticket Caps:** Enforce a maximum of 10 cumulative tickets per user per event to prevent seat hoarding.
- **Stateless Authentication:** Implement bcrypt-hashed passwords and stateless signed JWT tokens with Role-Based Access Control.
- **Digital Ticket Passes:** Generate formatted verification identifiers (EVT-YYYYMMDD-XXXX) and printable passes.
- **Automated Inventory Rollback:** Automatically restore event seats when reservations are cancelled.
- **Executive Telemetry:** Deliver interactive Recharts analytics and RFC 4180 CSV export pipelines.
- **Automated Verification:** Validate all endpoints and business rules with 77 automated test scenarios.

6. Proposed Solution

SkillOrbit delivers a clean, modern web application architecture: a React 19 Single Page Application styled with Tailwind CSS, an Express.js REST API with centralized error handling and database connection watchdogs, and a MongoDB Atlas database utilizing Mongoose 8.9 ODM. Atomic database operations prevent race conditions directly at the persistence layer, eliminating the overhead of distributed locks.

7. Functional Requirements

Attendee Requirements: Account registration, login, profile view, event browsing, category filtering, keyword search, concurrency-safe booking (1-10 seats), digital pass generation, personal dashboard, and self-service cancellation.

Administrator Requirements: Event creation, capacity adjustment with floor guards, soft cancellation for booked events, banner image upload, executive dashboard analytics, booking roster inspection, user management, and CSV report exports.

8. Non-Functional Requirements

- **Performance:** Sub-15ms response times on indexed database queries; production frontend bundle under 350 KB gzipped.
- **Concurrency Safety:** Deterministic seat allocations with zero overbooking under simultaneous requests.
- **Security:** Bcrypt hashing (10 salt rounds), signed JWT tokens, CORS origin whitelisting, and Mongoose query sanitization.
- **Reliability:** Connection watchdog middleware preventing requests to disconnected databases; centralized error handling.
- **Usability:** Responsive design across mobile, tablet, and desktop viewports; clean layout without white-label clutter.

9. Technology Stack

Layer	Technology	Version	Role & Architectural Purpose
Frontend UI	React	^19.0.0	Declarative component-driven user interface
Build Engine	Vite	^6.0.5	Development server and optimized production bundler
CSS Framework	Tailwind CSS	^3.4.17	Utility-first responsive design system
Icons	Lucide React	^0.468.0	Accessible, lightweight vector UI icons
Visualization	Recharts	^2.15.0	SVG analytics charting for admin dashboards
Client Router	React Router DOM	^7.1.0	Client-side routing and protected route guards
Backend Runtime	Node.js	v20 LTS / v22	Asynchronous JavaScript execution environment
Web Framework	Express.js	^4.19.2	RESTful HTTP API routing and middleware
Database Engine	MongoDB Atlas	7.0+	Cloud document database with atomic operators
ODM	Mongoose	^8.9.0	Schema validation, indexing, and lifecycle hooks
Authentication	jsonwebtoken	^9.0.2	Stateless HMAC SHA-256 signed bearer tokens
Cryptography	bcryptjs	^2.4.3	10-round salted password hashing
Testing	Jest & Supertest	^29.7.0	Automated integration and unit test runner

10. System Architecture

SkillOrbit uses a decoupled client-server architecture. The React 19 SPA communicates with the Express.js API via standard JSON HTTP requests. The backend is structured into modular layers: route definitions, controller business logic, authentication/role middleware, and Mongoose data models. Database requests are guarded by connection watchdog middleware, and unhandled errors pass cleanly through centralized error handlers.

11. Authentication and Authorization

User authentication uses signed JSON Web Tokens (JWT) containing user ID, role, and email claims. Passwords are hashed with bcryptjs (10 salt rounds) and excluded from queries by default (select: false). Role-Based Access Control (RBAC) is enforced at the API layer via protect and authorize('admin') middleware, and at the frontend layer via declarative <ProtectedRoute> route guards.

12. Event Management

Administrators manage events through full CRUD endpoints. When updating event capacity, the system enforces a floor guard: new capacity cannot be reduced below currently booked seats (capacity - availableSeats). When an administrator deletes an event with active bookings, the event status is set to 'cancelled' rather than hard-deleting the document, preserving historical records and ticket validity.

13. Event Discovery and Seat Availability

The event discovery catalog provides multi-parameter filtering across 10 categories, case-insensitive keyword search across four fields (title, description, venue, location), and upcoming status filtering. Cards display dynamic inventory badges: available seats, low availability warnings (under 15% remaining), and 'Sold Out' status with disabled booking controls.

14. Ticket Booking

Attendees can select between 1 and 10 tickets per reservation. The total price is computed from the snapshot ticket price. The system enforces a cumulative per-user limit: a single attendee cannot hold more than 10 confirmed tickets for any given event, preventing bulk ticket hoarding.

15. Concurrency-Safe Booking

To prevent overselling under high concurrency without the overhead of heavy distributed locks, SkillOrbit executes atomic conditional updates directly within MongoDB:

```
// Atomic Concurrency-Safe Decrement in bookingController.js
const event = await Event.findOneAndUpdate(
  {
    _id: eventId,
    status: 'published',
    availableSeats: { $gte: seats } // Atomic guard
  },
  {
    $inc: { availableSeats: -seats } // Atomic decrement
  },
  { new: true }
);
if (!event) {
  return res.status(400).json({ success: false, message: 'Insufficient seats available.' });
}
```

Because MongoDB serializes single-document mutations atomically, two concurrent requests competing for the same remaining seat cannot both satisfy the `$gte` condition. One succeeds and receives the updated document; the other fails and receives null, preventing overbooking.

16. Booking Cancellation

Attendees can cancel active bookings via `PUT /api/bookings/:id/cancel`. The endpoint verifies ownership, updates booking status to 'cancelled', and executes an atomic rollback (`Event.findByIdAndUpdate(..., { $inc: { availableSeats: seatsBooked } })`) to restore the released seats to the event inventory immediately.

17. Digital Booking Confirmation / Ticket Generation

Upon confirmation, the system creates a digital boarding pass with an algorithmically formatted reference code (EVT-YYYYMMDD-XXXX). The pass displays attendee details, event schedules, venue info, and pricing snapshots, with one-click print/PDF support formatted for physical check-in.

18. Admin Operations and Analytics

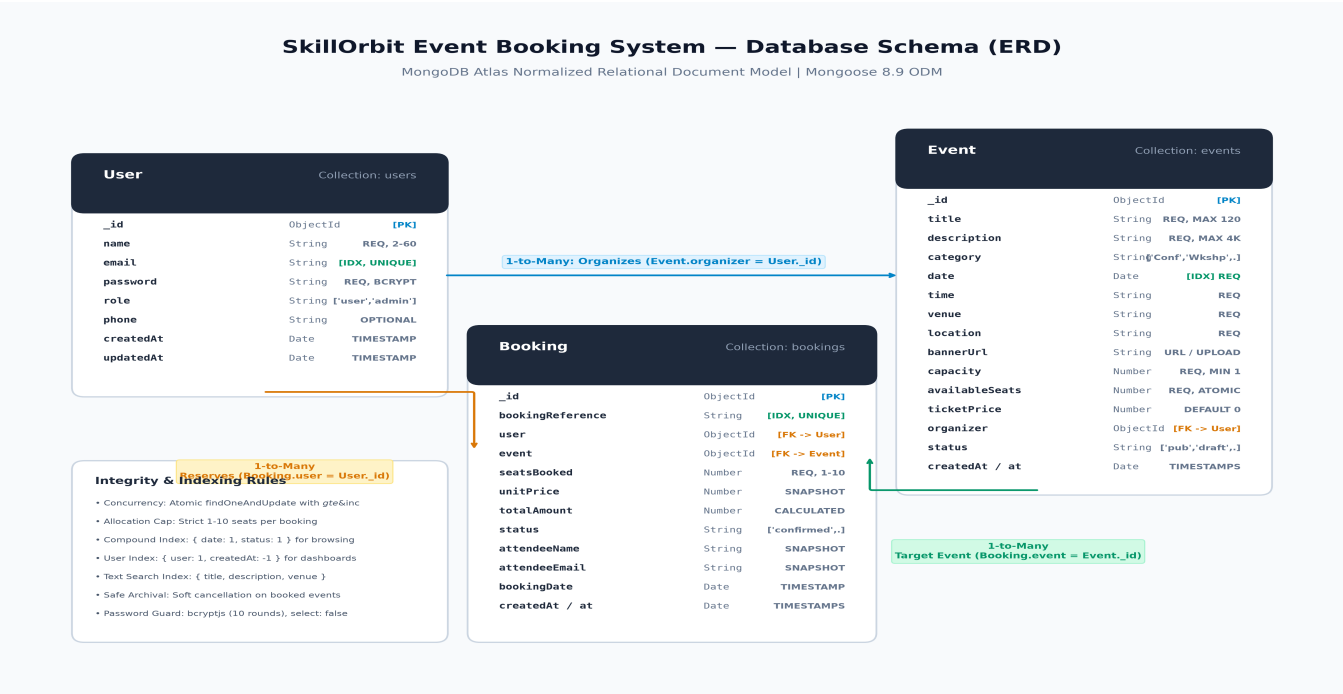
The administrative dashboard (`/admin`) provides real-time telemetry: total events, active attendees, tickets booked, and gross revenue. Interactive Recharts components display category breakdowns (donut chart) and monthly registration trends (area chart) over the past six months, alongside recent booking feeds and user management tables.

19. CSV Export and Reporting

SkillOrbit provides RFC 4180 compliant CSV export pipelines using `json2csv`. Administrators can download complete booking rosters (`GET /api/reports/bookings/csv`) and event occupancy summaries (`GET /api/reports/events/csv`) for venue check-ins and capacity audits.

20. Database Design and Data Models

The database schema consists of three collections: users, events, and bookings. Relational references link documents, while critical pricing and attendee fields are snapshotted inside bookings for historical consistency.



21. Frontend Architecture and User Experience

Built with React 19, Vite, and Tailwind CSS, the user interface follows a clean, professional design language. It avoids generic design patterns such as purple gradients, pill buttons, fake counters, stock photos, or distracting animations. The palette emphasizes deep slate and navy accents (#0F172A, #1E293B, #0284C7) with emerald status indicators (#059669). A centralized AuthContext manages authentication state, while a custom toast notification system provides feedback.

22. Security Architecture

- **Password Security:** 10-round bcrypt salted hashing; passwords excluded from queries by default (select: false).
- **Token Authentication:** Stateless signed JWTs with expiration; role validation middleware on protected routes.
- **Injection Defense:** Mongoose schema casting and parameterized queries mitigate NoSQL injection risks.
- **CORS Whitelisting:** Restricts cross-origin requests to authorized development and production origins.
- **Authorization Guards:** Attendees cannot view or cancel bookings belonging to other users.

23. Deployment and Infrastructure

- **Frontend Hosting:** Deployed on Vercel Edge Network: <https://skillorbit-event-booking-system.vercel.app/>
- **Backend API Hosting:** Deployed on Render Web Service: <https://skillorbit-backend.onrender.com>
- **Database Hosting:** MongoDB Atlas Managed Cloud Cluster (TLS encryption, automated backups, IP allowlisting).
- **Health Check Endpoint:** GET /api/health verifies API uptime and database connectivity.

24. Performance Optimization

Targeted compound B-tree indexes ({ date: 1, status: 1 } and { user: 1, createdAt: -1 }) ensure sub-15ms query execution. Selective denormalization of pricing data eliminates multi-collection joins during booking lookups. Frontend production assets are bundled via Vite with tree-shaking, keeping total bundle size under 350 KB gzipped.

25. Verification and Testing

Test Suite	File Path	Key Functional Scenarios Verified	Pass Rate
System & Health	tests/health.test.js	API uptime, DB connectivity status, headers	5 / 5 (100%)
Authentication	tests/auth.test.js	Registration, duplicate emails, bcrypt hash, JWT issue	14 / 14 (100%)
Events Management	tests/events.test.js	Catalog search, category tabs, capacity floor, soft-delete	22 / 22 (100%)
Booking Engine	tests/bookings.test.js	Atomic decrement, overselling rejection, cancellation rollback	24 / 24 (100%)
Admin & Reports	tests/admin.test.js	Aggregations, monthly trends, user directory, CSV export	12 / 12 (100%)
Overall Quality	5 Test Suites	Complete integration & unit test suite	77 / 77 (100%)

26. Project Structure

The project is organized into modular frontend and backend packages: `backend/src/` contains controllers, middleware, models, routes, and config; `frontend/src/` contains components, context, pages, and services; and `docs/` contains full technical specifications and reports.

27. Setup and Installation

Backend Setup: Navigate to `backend/`, run `npm install`, configure `.env` (`PORT`, `MONGODB_URI`, `JWT_SECRET`, `CLIENT_URL`), and run `npm run dev` (or `npm test` to execute the test suite).

Frontend Setup: Navigate to `frontend/`, run `npm install`, configure `VITE_API_URL`, and run `npm run dev`.

28. Limitations

- Outbound SMTP email delivery is not connected; digital passes are generated directly in the browser interface.
- External payment processor checkout flows (Stripe/Razorpay) are not connected; ticket prices are recorded as snapshot data.
- Hardware barcode/QR scanning integration for turnstiles is not implemented.
- Event dates are scheduled in UTC and displayed in local browser timezone without multi-timezone conversions.

29. Future Scope

- Integration of transactional email services (SendGrid/AWS SES) for automatic PDF pass delivery.
- Payment gateway webhook integration (Razorpay/Stripe) to support paid ticket checkouts.
- Mobile wallet pass generation (`.pkpass` for Apple Wallet and Google Wallet).
- Automated waitlist management to allocate seats when cancellations occur.

30. References

1. React Documentation: Official React 19 Reference & Hooks Guide. <https://react.dev/>
2. Express.js Documentation: Routing, Middleware, and Centralized Error Handling. <https://expressjs.com/>
3. MongoDB Manual: Atomicity, Concurrency Control, and Operators (`$inc`, `$gte`). <https://www.mongodb.com/docs/manual/>
4. Mongoose ODM: Schema Validation, Population, and Middleware Hooks. <https://mongoosejs.com/docs/>
5. Tailwind CSS Documentation: Utility-First CSS Framework & Design System. <https://tailwindcss.com/docs>
6. IETF RFC 7519: JSON Web Token (JWT) Standard Specification. <https://datatracker.ietf.org/doc/html/rfc7519>
7. IETF RFC 4180: Common Format and MIME Type for Comma-Separated Values (CSV). <https://datatracker.ietf.org/doc/html/rfc4180>
8. Jest Testing Framework: JavaScript Automated Testing Platform. <https://jestjs.io/>
9. Supertest Documentation: High-Level HTTP Testing Abstraction for Node.js. <https://github.com/ladjs/supertest>